SOFTWARE REQUIREMENTS SPECIFICATION

for

Development of an Intelligent Course and Resource Recommendation System

Version 1.0

Prepared in accordance with IEEE 830 SRS Guidelines

Date: 28 July 2026

Software Engineering — Laboratory Assignment 1

Table of Contents

Table of Contents 2
0 Document Control 4
0.1 Revision History 4
0.2 Approval 4
1 Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 Definitions, Acronyms and Abbreviations 6
1.4 References 6
1.5 Document Overview 6
2 Overall Description 8
2.1 Product Perspective 8
2.2 Product Functions 8
2.3 User Classes and Characteristics 8
2.4 Operating Environment 9
2.5 Design and Implementation Constraints 9
2.6 Assumptions and Dependencies 9
3 System Features (Functional Requirements) 10
3.1 Feature: User Registration and Login (FR-1) 10
3.2 Feature: Profile Management (FR-2) 11
3.3 Feature: Recommendation Engine (FR-3) 12
3.4 Feature: Feedback Loop (FR-4) 13
3.5 Feature: Catalogue / Administration (FR-5) 13
3.6 Summary of Functional Requirements 14
4 External Interface Requirements 15
4.1 User Interfaces 15
4.2 Hardware Interfaces 15
4.3 Software Interfaces 15
4.4 Communication Interfaces 15
5 System Architecture and Design 16
5.1 Technology Stack 16
5.2 High-Level Architecture 16
5.3 Database Design (Entity-Relationship Overview) 16

5.4 Class Diagram (Description) 17
5.5 Use Case Diagram (Description) 18
5.6 Sequence Diagram (Description) — "Request Recommendations" Flow 18
5.7 Activity Diagram (Description) — Recommendation Algorithm Logic 19
6 Non-Functional Requirements 20
6.1 Performance (NFR-1) 20
6.2 Security 20
6.3 Extensibility (NFR-2) 20
6.4 Reliability and Availability 20
6.5 Usability 20
6.6 Maintainability 20
6.7 Portability 21
7 Data Requirements 22
7.1 Mock Resource Catalogue 22
7.2 Dummy Users and Interaction Histories 22
8 Project Deliverables 24
9 Test Plan and Representative Test Cases 25
9.1 Testing Approach 25
9.2 Representative Test Cases 25
9.3 Acceptance Criteria 26
10 Appendices 27
10.1 Appendix A — Glossary 27
10.2 Appendix B — Requirements Traceability Matrix 27
10.3 Appendix C — Sprint Plan Outline (Agile Implementation) 27
10.4 Appendix D — Assumptions Log 28

0 Document Control

0.1 Revision History

Version	Date	Author	Description
0.1	Draft	Project Team	Initial draft of problem statement and scope
0.5	Draft	Project Team	Added functional and non-functional requirements
1.0	Final	Project Team	Reviewed and finalized SRS for submission

0.2 Approval

Role	Name	Signature	Date
Project Guide / Course Instructor	_____	_____	_____
Team Lead	_____	_____	_____

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and structured description of the functional and non-functional requirements for the "Intelligent Course and Resource Recommendation System" (hereafter referred to as ICRRS or "the System"). The purpose of this document is to serve as the single point of reference for the development team, the project guide/instructor, and any stakeholder involved in evaluating, designing, implementing, or testing the system. It defines what the system will do, the constraints under which it must operate, and the quality attributes it must satisfy, without prescribing a specific internal implementation.

This document is intended to be read by software engineering students undertaking the project, the course instructor evaluating the submission, and any future developers who may extend the system. It assumes a working knowledge of standard software engineering terminology, relational databases, and object-oriented design.

1.2 Scope

The system to be produced is a mini educational-technology application named the Intelligent Course and Resource Recommendation System. In the digital age, learners are exposed to an overwhelming abundance of educational resources — video lectures, articles, MOOCs, textbooks, and interactive tutorials — spread across many platforms. This abundance leads to "information overload," where a learner struggles to identify the courses or materials that are most relevant to their current skill level, career objective, or immediate academic need.

Currently, learners resolve this problem manually: they search generic engines or browse catalogues, a process that is time-consuming and frequently yields disjointed, redundant, or inappropriate material. The ICRRS is being engineered to replace this manual search with a structured, automated recommendation workflow. The system will:

- Maintain a structured catalogue of learning resources (courses, tutorials, articles) tagged with metadata such as topic, difficulty level, and prerequisites.
- Maintain a model of each user's skills, interests, goals, and interaction history.
- Apply a matching/ranking algorithm to map user profiles against the resource catalogue and produce a ranked list of the top five most relevant resources.
- Collect explicit feedback (star ratings) from users on suggested resources and log this feedback so that future recommendation cycles can be improved.
- Be modelled using Object-Oriented Analysis and Design (OOAD) principles and documented with standard UML diagrams to ensure the resulting architecture is scalable and maintainable.
- Be delivered as a Minimum Viable Product (MVP) using an iterative, Agile (Scrum-style) development process across a small number of sprints.

The scope of the initial release (MVP) is deliberately bounded: it targets a single learner-facing recommendation workflow with a mock/dummy dataset, rather than integration with live third-party

course providers. Payment processing, live video streaming, social/community features, and mobile native applications are explicitly out of scope for this release and are listed in Section 2.6 as candidate future enhancements.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification — this document.
MVP	Minimum Viable Product — the smallest feature set that delivers usable value.
UML	Unified Modeling Language — a standard notation for visualizing OOAD artifacts.
OOAD	Object-Oriented Analysis and Design.
CLI	Command-Line Interface.
CRUD	Create, Read, Update, Delete — the four basic persistence operations.
ERD	Entity-Relationship Diagram — a data-modeling notation for relational databases.
API	Application Programming Interface.
Recommendation Engine	The core module that ranks resources against a user profile.
Resource	A generic term for a course, tutorial, article, or other learning material stored in the catalogue.
User Profile	The structured record of a learner's skills, goals, and preferences.
Feedback Loop	The mechanism by which user ratings are captured and fed back to refine future recommendations.
Strategy Pattern	A behavioural design pattern that lets an algorithm's behaviour be selected at run time, used here to keep the engine extensible.
Sprint	A fixed, short iteration (typically one to two weeks) in Agile/Scrum development.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 1016, IEEE Recommended Practice for Software Design Descriptions.
- Object Management Group (OMG), Unified Modeling Language Specification, Version 2.5.
- Gamma, Helm, Johnson, Vlissides — Design Patterns: Elements of Reusable Object-Oriented Software (Strategy pattern reference).
- Course Lab Assignment brief: "Development of an Intelligent Course and Resource Recommendation System" (project problem statement, as supplied by the course instructor).

1.5 Document Overview

The remainder of this document is organised as follows. Section 2 gives an overall, non-technical description of the product, its users, and the constraints it operates under. Section 3 enumerates the detailed functional requirements as system features, expressed as fully-dressed use cases. Section 4 specifies external interface requirements. Section 5 describes the proposed system architecture, technology stack, database design, and the four required UML diagrams (use case, class, sequence, and activity). Section 6 specifies non-functional (quality) requirements. Section 7 details data requirements including the mock dataset. Section 8 lists project deliverables. Section 9 provides the test plan and representative test cases. Section 10 contains appendices, including a glossary and a requirements traceability matrix.

2 Overall Description

2.1 Product Perspective

The ICRRS is a new, self-contained academic mini-project; it is not a component of, nor does it replace, any existing production system. Conceptually it sits between a learner and a catalogue of learning resources, in the same role that a recommendation engine occupies in commercial platforms such as streaming or e-commerce services, but scoped down to an educational-technology context and to the size appropriate for a semester-long software engineering laboratory assignment.

At a high level the product perspective can be summarised as three cooperating layers: a persistence layer (MySQL) that stores users, resources, and interaction/rating history; a core logic layer (Python or C++) that implements profile matching, ranking, and the feedback loop; and a presentation layer (a simple web UI or a CLI) through which the learner interacts with the system. This layered separation is chosen deliberately so that the recommendation algorithm can be replaced or extended (see the Strategy pattern requirement in Section 3 and Section 6.3) without requiring changes to the persistence or presentation layers.

2.2 Product Functions

At a summary level, the system performs the following major functions; each is elaborated into detailed functional requirements in Section 3.

- Register new users and authenticate returning users, maintaining a logged-in session.
- Allow a user to create and update a profile describing current skills and target learning goals.
- Maintain a catalogue of at least 50–100 learning resources with tags, difficulty levels, and prerequisites.
- Compute and present a ranked list of the top five recommended resources for a given user profile in under two seconds.
- Capture a 1–5 star rating (feedback) from the user for any recommended resource and persist it against both the user and the resource.
- Use logged feedback as an input signal so that future recommendation runs can be improved over time.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Learner (Primary Actor)	A student or self-learner who registers, builds a profile, receives recommendations, and rates them.	Low to moderate — general computer literacy only.
System Administrator	Maintains the resource catalogue (adds/edits/removes courses), monitors system health, and reviews aggregate feedback.	Moderate to high — comfortable with structured data entry / basic SQL.

User Class	Description	Technical Expertise
Developer / Maintainer	Extends the recommendation engine, adds new algorithms via the Strategy pattern, and maintains the codebase.	High — proficient in the chosen backend language and OOAD.

2.4 Operating Environment

The MVP is designed to run in a standard development and demonstration environment rather than a production cloud deployment. The following environment is assumed:

- Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS — the stack is cross-platform.
- Database Server: MySQL 8.0 or later, running locally or on a lightweight cloud instance.
- Backend Runtime: Python 3.10+ (if the Python option is chosen) or a C++17-compliant compiler/toolchain (if the C++ option is chosen).
- Frontend (optional): any evergreen web browser (Chrome, Firefox, Edge) for the HTML/CSS/JavaScript interface, or a standard terminal emulator for the CLI variant.
- Modeling Tools: Lucidchart, Draw.io, or StarUML for producing/maintaining the UML diagrams referenced in Section 5.

2.5 Design and Implementation Constraints

- The system must be modelled using Object-Oriented principles; the class design must map cleanly onto the relational schema in MySQL.
- The recommendation algorithm must be encapsulated behind a stable interface (Strategy pattern) so alternative algorithms can be substituted without modifying calling code.
- The suggestion algorithm must return a ranked list of five resource IDs in under two seconds under the expected MVP data volumes (50–100 resources, tens of dummy users).
- Development must follow an Agile process delivered in iterative sprints, each ending with a demonstrable increment.
- The project must be reproducible by a grader: setup steps, seed data, and run instructions must be documented alongside the source code.

2.6 Assumptions and Dependencies

- It is assumed that a mock/dummy dataset (not a live, licensed course catalogue) is acceptable for grading purposes, per the project brief.
- It is assumed the team has access to a MySQL instance for the duration of development and demonstration.
- The following are explicitly out of scope for the MVP and are dependencies for any future release: payment/e-commerce integration, integration with live third-party MOOC providers, native mobile applications, real-time chat/social features, and multi-language localisation.
- It is assumed that user authentication for the MVP may be simplified (e.g., username/password without third-party OAuth) unless the instructor specifies otherwise.

3 System Features (Functional Requirements)

This section decomposes the four functional requirement areas identified in the problem statement — Registration/Login, Profile Management, the Recommendation Engine, and the Feedback Loop — into fully-dressed use cases, each with a unique identifier for traceability (see Section 10.2).

3.1 Feature: User Registration and Login (FR-1)

Description: The system must securely register new users and authenticate returning users, maintaining their session for the duration of use.

Use Case ID	UC-01
Use Case Name	Register New Account
Actor(s)	Learner
Priority	High
Precondition(s)	The learner has a valid email address and is not already registered.
Trigger	Learner selects "Sign Up" from the landing screen.
Main Flow	1. Learner provides name, email, and password. 2. System validates the email format and password strength. 3. System checks the email is not already registered. 4. System hashes the password and creates a User record. 5. System creates an empty Profile linked to the new User. 6. System logs the learner in and redirects to Profile Setup.
Alternate / Exception Flow	3a. Email already exists → system displays an error and offers a login link. 2a. Password fails strength rules → system displays specific validation feedback.
Postcondition(s)	A new user record exists in the database; the learner is logged in.

Use Case ID	UC-02
Use Case Name	Log In
Actor(s)	Learner
Priority	High
Precondition(s)	The learner already holds a registered account.
Trigger	Learner selects "Log In" and submits credentials.
Main Flow	1. Learner enters email and password. 2. System verifies credentials against stored password hash. 3. System creates a session/token. 4. System redirects the learner to the Dashboard.
Alternate / Exception Flow	2a. Credentials invalid → system displays a generic "invalid email or password" message (to avoid revealing which field is wrong) and increments a failed-

	attempt counter. 2b. Account locked after repeated failures → system displays a lockout message.
Postcondition(s)	A valid session is created for the learner.

3.2 Feature: Profile Management (FR-2)

Description: Registered users must be able to input and later update their current skills and target learning goals; this profile is the primary input to the Recommendation Engine.

Use Case ID	UC-03
Use Case Name	Create / Edit Profile
Actor(s)	Learner
Priority	High
Precondition(s)	Learner is logged in.
Trigger	Learner opens "My Profile" and edits fields.
Main Flow	1. Learner selects current skills from a tag list (or adds free-text skills). 2. Learner selects one or more target goals / career tracks. 3. Learner optionally sets a preferred difficulty level (Beginner / Intermediate / Advanced). 4. System validates that at least one skill or goal is present. 5. System saves the Profile and marks it as the active profile for recommendation.
Alternate / Exception Flow	4a. No skills or goals entered → system blocks save and prompts the learner to add at least one.
Postcondition(s)	The Profile record (skills, goals, preferred difficulty) is created or updated and timestamped.

Use Case ID	UC-04
Use Case Name	View Learning History
Actor(s)	Learner
Priority	Medium
Precondition(s)	Learner is logged in and has at least one past interaction.
Trigger	Learner opens "My History".
Main Flow	1. System retrieves all past recommendations and ratings for the logged-in user. 2. System displays them in reverse-chronological order with resource title, date, and rating.
Alternate / Exception Flow	1a. No history exists → system displays an empty-state message inviting the learner to request a recommendation.
Postcondition(s)	None (read-only use case).

3.3 Feature: Recommendation Engine (FR-3)

Description: The core feature of the system. It processes the active user profile and the resource catalogue and outputs a ranked list of the top five resource IDs, returned in under two seconds (see NFR-1, Section 6.1).

Use Case ID	UC-05
Use Case Name	Request Recommendations
Actor(s)	Learner
Priority	High (core feature)
Precondition(s)	Learner is logged in and has a saved Profile with at least one skill or goal.
Trigger	Learner clicks "Get Recommendations" on the Dashboard.
Main Flow	1. System loads the learner's active Profile. 2. System loads eligible Resources from the catalogue (filtered by prerequisite eligibility). 3. The active Recommendation Strategy scores each eligible Resource against the Profile (matching tags, difficulty fit, and, where available, prior ratings from similar users). 4. System sorts Resources by score, descending, and selects the top five. 5. System returns the ranked list to the UI and stores a Recommendation log entry (user, timestamp, resource IDs, algorithm version).
Alternate / Exception Flow	2a. Fewer than five eligible resources exist → system returns as many as are eligible and displays a note. 3a. Profile has no matchable tags → system falls back to a popularity-based ranking and flags the result as a "cold start" recommendation.
Postcondition(s)	A ranked list of five Resource IDs is displayed and a Recommendation record is logged for traceability.

Use Case ID	UC-06
Use Case Name	Filter Recommendations by Difficulty
Actor(s)	Learner
Priority	Medium — directly testable ("if user selects Beginner, verify no Advanced courses are suggested").
Precondition(s)	A recommendation list has already been generated (UC-05).
Trigger	Learner selects a difficulty filter (e.g., "Beginner only").
Main Flow	1. Learner selects a difficulty level from the filter control. 2. System re-queries the ranked list restricted to that difficulty. 3. System re-displays the filtered top five (or fewer).
Alternate / Exception Flow	2a. No resources match the filter → system displays an empty-state message.

Postcondition(s)	The displayed list is limited to the selected difficulty and never includes a higher difficulty than requested.

3.4 Feature: Feedback Loop (FR-4)

Description: Users must be able to rate a suggested resource from 1 to 5 stars; the system logs this rating and uses it to improve future suggestions.

Use Case ID	UC-07
Use Case Name	Rate a Recommended Resource
Actor(s)	Learner
Priority	High
Precondition(s)	Learner has received at least one recommendation (UC-05) and has engaged with the resource.
Trigger	Learner clicks a star-rating control next to a recommended resource.
Main Flow	1. Learner selects a value from 1 to 5 stars. 2. System validates the value is within range and that the learner has not already rated this specific recommendation instance. 3. System persists the Rating and links it to the Resource and the User. 4. System updates the Resource's aggregate average rating. 5. System marks the Rating as an input signal for the next recommendation run.
Alternate / Exception Flow	2a. Learner attempts to rate the same recommendation instance twice → system updates the existing rating instead of creating a duplicate.
Postcondition(s)	A Rating record (user, resource, score, timestamp) is persisted and linked to the originating Recommendation log entry.

3.5 Feature: Catalogue / Administration (FR-5)

Description: Although optional for a minimal MVP, a basic administrative capability is included so the mock dataset (Section 7) can be seeded and maintained without direct database access, and so the design demonstrates full CRUD coverage.

Use Case ID	UC-08
Use Case Name	Manage Resource Catalogue
Actor(s)	System Administrator
Priority	Medium
Precondition(s)	Administrator is logged in with an Admin role.
Trigger	Administrator opens the "Manage Catalogue" screen.
Main Flow	1. Administrator adds a new Resource (title, tags, difficulty, prerequisites) or selects an existing Resource to edit or remove. 2. System validates required

	fields (title, at least one tag, difficulty level). 3. System persists the change to the catalogue.
Alternate / Exception Flow	2a. Required field missing → system blocks save and highlights the missing field. 1a. Administrator attempts to delete a Resource with existing Ratings → system asks for confirmation before cascading the delete.
Postcondition(s)	The Resource catalogue reflects the added, edited, or removed entry.

3.6 Summary of Functional Requirements

ID	Requirement	Related Use Case(s)	Priority
FR-1	The system shall allow secure registration and login of users.	UC-01, UC-02	High
FR-2	The system shall allow users to create and update a profile of skills and goals.	UC-03, UC-04	High
FR-3	The system shall generate a ranked list of the top 5 recommended resources for a user profile.	UC-05, UC-06	High
FR-4	The system shall allow users to rate a recommended resource from 1 to 5 stars and log the rating.	UC-07	High
FR-5	The system shall provide administrative CRUD operations on the resource catalogue.	UC-08	Medium
FR-6	The system shall exclude resources whose difficulty exceeds the user's selected filter/preference.	UC-06	Medium
FR-7	The system shall fall back to a popularity-based ranking when a profile has no matchable tags (cold start).	UC-05	Medium
FR-8	The system shall prevent duplicate ratings for the same recommendation instance by updating the existing rating.	UC-07	Low

4 External Interface Requirements

4.1 User Interfaces

For the MVP, two interface options are permitted, selected according to the team's chosen emphasis (see Section 5.1):

- Web UI: a lightweight HTML/CSS/JavaScript single-page or multi-page interface providing Registration/Login screens, a Profile editor, a Dashboard displaying the top-5 recommendation cards, a star-rating control, and (for administrators) a Catalogue management screen.
- CLI: a menu-driven command-line interface offering the equivalent operations as numbered menu options, suitable when the project's focus is strictly backend architecture and modelling.

Regardless of the chosen option, all user-facing screens/menus must present clear, unambiguous labels, validate input before submission, and surface system errors (e.g., invalid credentials, missing profile fields) as human-readable messages rather than raw stack traces or SQL errors.

4.2 Hardware Interfaces

The MVP has no dedicated hardware interface requirements. It is expected to run on standard commodity development hardware (a single laptop/desktop for local development and demonstration) with no requirement for specialised peripherals, sensors, or embedded devices.

4.3 Software Interfaces

Interface	Description
MySQL Database	Primary persistence store, accessed via a standard driver (e.g., mysql-connector-python, or a C++ MySQL connector library) using parameterised queries.
Backend Runtime (Python or C++)	Hosts the User, Profile, Resource, Recommendation, and Rating classes and the Recommendation Engine's Strategy implementations.
Frontend Runtime	A browser executing the HTML/CSS/JavaScript client, communicating with the backend via a simple HTTP/JSON API (if the web option is chosen), or the terminal executing the CLI binary directly against the backend library.
UML Modelling Tool Export	Diagrams produced in Lucidchart / Draw.io / StarUML are exported as image or PDF assets and embedded in the project report; no runtime dependency.

4.4 Communication Interfaces

If the web UI option is selected, the frontend and backend communicate over HTTP using JSON request/response payloads (a minimal internal REST-style API, e.g., POST /login, GET /recommendations, POST /ratings). No external network communication with third-party services is required for the MVP; all data originates from the local mock database described in Section 7.

5 System Architecture and Design

5.1 Technology Stack

Layer	Recommended Technology	Rationale
Database	MySQL	Strong relational modelling for Users, Resources, and the many-to-many interaction/rating history between them; mature tooling and wide familiarity.
Backend / Core Logic	Python (with optional lightweight ML libraries) OR C++	Python favours rapid iteration and simple data-analysis/ML experimentation for scoring; C++ favours a rigorous, high-performance OOP implementation of the engine's core patterns.
Frontend (optional for MVP)	HTML / CSS / JavaScript, or a CLI	A minimal web client keeps the demonstration accessible; a CLI keeps the team's effort focused on backend architecture if that is the project's emphasis.
Modelling	Lucidchart, Draw.io, or StarUML	Produces the UML diagrams required as deliverables (Section 5.4–5.7 and Section 8).

5.2 High-Level Architecture

The system follows a layered architecture with a clear separation of concerns, which both satisfies the extensibility non-functional requirement (Section 6.3) and maps naturally onto the required UML class diagram (Section 5.4):

- Presentation Layer — the Web UI or CLI; responsible only for input capture and rendering output, with no business logic.
- Application / Service Layer — orchestrates use cases (e.g., RecommendationService, ProfileService, AuthService), coordinating calls between the Presentation Layer and the Domain Layer.
- Domain Layer — the core object model (User, Profile, Resource, Recommendation, Rating) plus the Recommendation Engine, implemented with the Strategy pattern so that alternative scoring algorithms (e.g., a simple tag-overlap strategy versus a weighted collaborative-filtering strategy) can be swapped without modifying the Service Layer.
- Persistence Layer — a thin data-access layer (Repository pattern) that translates domain objects to and from MySQL tables, isolating SQL from business logic.

This layering directly supports NFR-2 (Extensibility, Section 6.3): a new recommendation algorithm is added by implementing a new class that satisfies the RecommendationStrategy interface, without any change to RecommendationService, the UI, or the persistence schema.

5.3 Database Design (Entity-Relationship Overview)

The relational schema is derived directly from the domain classes described in Section 5.4. The core tables and their relationships are summarised below; primary keys are denoted PK and foreign keys FK.

Table	Key Columns	Notes
users	user_id (PK), name, email, password_hash, role, created_at	role distinguishes Learner vs Administrator.
profiles	profile_id (PK), user_id (FK), skills, goals, preferred_difficulty, updated_at	One-to-one with users (an active profile per user).
resources	resource_id (PK), title, tags, difficulty_level, prerequisites, created_at	The mock catalogue of 50–100 dummy courses (Section 7).
recommendations	recommendation_id (PK), user_id (FK), resource_id (FK), rank, algorithm_version, generated_at	One row per (user, resource) pair returned in a recommendation run; supports the traceability required by the feedback loop.
ratings	rating_id (PK), recommendation_id (FK), user_id (FK), resource_id (FK), score, rated_at	score constrained to the integer range 1–5.

Relationships: a user has exactly one active profile (1:1 users→profiles); a user may receive many recommendations over time and a resource may appear in many recommendations (M:N users↔resources, realised through the recommendations table); a rating is always linked back to the specific recommendation instance it originated from, which is what allows the feedback loop (FR-4) to attribute a rating to a particular algorithm run.

5.4 Class Diagram (Description)

The class diagram models the Domain Layer described in Section 5.2. Because this document is a textual specification, the diagram's structure is described here in tabular form; the corresponding graphical UML Class Diagram is produced separately using the modelling tool selected in Section 5.1 and is included as a project deliverable (Section 8).

Class	Key Attributes	Key Operations	Relationships
User	userId, name, email, passwordHash, role	register(), login(), logout()	1 — 1 Profile; 1 — * Rating
Profile	profileId, skills[], goals[], preferredDifficulty	updateSkills(), updateGoals(), setDifficulty()	belongs to 1 User
Resource	resourceId, title, tags[], difficultyLevel, prerequisites[]	matchesProfile(profile), addTag(), removeTag()	* — * Recommendation
RecommendationEngine	strategy: RecommendationStrategy	generateRecommendations(profile, catalogue): List<Resource>	uses 1 RecommendationStrategy (Strategy pattern)

Class	Key Attributes	Key Operations	Relationships
RecommendationStrategy (interface)	—	score(profile, resource): double	implemented by TagOverlapStrategy, PopularityStrategy, etc.
Recommendation	recommendationId, userId, resourceId, rank, algorithmVersion, generatedAt	logResult()	links 1 User to 1 Resource
Rating	ratingId, recommendationId, score, ratedAt	validateScore()	belongs to 1 Recommendation
AdminService	—	addResource(), editResource(), removeResource()	manages Resource catalogue

5.5 Use Case Diagram (Description)

The Use Case Diagram captures the interactions between the two primary actors — Learner and System Administrator — and the system boundary. It comprises the following actor-to-use-case associations, each elaborated fully in Section 3:

- Learner: Register New Account (UC-01), Log In (UC-02), Create/Edit Profile (UC-03), View Learning History (UC-04), Request Recommendations (UC-05), Filter Recommendations by Difficulty (UC-06), Rate a Recommended Resource (UC-07).
- System Administrator: Log In (UC-02, shared), Manage Resource Catalogue (UC-08).
- Include relationship: UC-05 (Request Recommendations) «includes» a system-internal step of loading the active Profile, which depends on UC-03 having been completed at least once.
- Extend relationship: UC-06 (Filter by Difficulty) «extends» UC-05, since filtering is an optional refinement of an already-generated recommendation list.

5.6 Sequence Diagram (Description) — "Request Recommendations" Flow

This sequence diagram traces the data flow for UC-05, the system's core feature, across five participants: Learner, UI, RecommendationService, RecommendationEngine, and the Database.

1. Learner clicks "Get Recommendations" on the UI.
2. UI sends a getRecommendations(userId) call to RecommendationService.
3. RecommendationService requests the active Profile for userId from the Database.
4. RecommendationService requests the eligible Resource catalogue from the Database.
5. RecommendationService passes the Profile and catalogue to RecommendationEngine.generateRecommendations().
6. RecommendationEngine delegates scoring of each Resource to the configured RecommendationStrategy.score().

7. RecommendationEngine sorts scored Resources and returns the top five to RecommendationService.
8. RecommendationService writes a Recommendation log row to the Database for each of the five results.
9. RecommendationService returns the ranked list to the UI, which renders it to the Learner.

This flow must complete within the two-second response-time budget specified in NFR-1 (Section 6.1); the sequence diagram deliverable should annotate the RecommendationEngine activation bar with this constraint.

5.7 Activity Diagram (Description) — Recommendation Algorithm Logic

The activity diagram documents the internal decision logic of the Recommendation Engine, covering both the normal path and the cold-start fallback (FR-7):

10. Start.
11. Load active Profile for the user.
12. Decision: Does the Profile contain at least one matchable skill/goal tag?
13. If NO → follow the cold-start branch: rank Resources by overall popularity (average rating × number of ratings) and flag the result set as "cold start".
14. If YES → follow the personalised branch: for each eligible Resource, compute a score combining tag overlap with the Profile, difficulty fit, and (where available) a collaborative signal from similar users' ratings.
15. Filter out any Resource whose prerequisites are not satisfied by the Profile's skills.
16. Sort remaining Resources by score, descending.
17. Select the top five.
18. Persist a Recommendation log entry for each selected Resource.
19. End — return ranked list to caller.

6 Non-Functional Requirements

6.1 Performance (NFR-1)

The Recommendation Engine shall return the ranked top-five result set in under 2 seconds, measured from the moment the request is received by RecommendationService to the moment the response is returned to the UI, under the expected MVP data volume (50–100 resources and a modest number of dummy users). This shall be verified with the test cases in Section 9 (see TC-PERF-01).

6.2 Security

- Passwords shall never be stored in plain text; they shall be hashed using a standard salted hashing algorithm before persistence.
- All database queries shall use parameterised statements to prevent SQL injection.
- Session tokens (if a web API is used) shall expire after a period of inactivity and shall not be exposed in URLs.
- Administrative operations (UC-08) shall be restricted to users whose role attribute is Administrator.

6.3 Extensibility (NFR-2)

The codebase shall be designed using the Strategy pattern (see Section 5.2 and 5.4) so that new recommendation algorithms can be added by implementing the RecommendationStrategy interface, without modifying RecommendationService, the persistence schema, or the user interface. This requirement is directly traceable to the problem statement's stated design constraint.

6.4 Reliability and Availability

- The system shall handle invalid or missing input (e.g., an empty profile, a malformed rating value) gracefully, returning a descriptive error rather than crashing.
- Database writes for Recommendation and Rating records shall be atomic — a partially written recommendation run shall not be visible to subsequent reads.
- As an academic MVP, formal uptime SLAs are not required; however, the system shall recover cleanly (no data corruption) after an abrupt process restart.

6.5 Usability

- A first-time user shall be able to complete registration, profile creation, and obtain a first recommendation within 5 minutes without external instructions, beyond in-app labels/prompts.
- Error messages shall be written in plain language and shall never expose raw SQL or stack traces to end users.
- The star-rating control shall provide immediate visual confirmation of the selected rating.

6.6 Maintainability

- Source code shall be organised by layer (Presentation, Service, Domain, Persistence) as described in Section 5.2, with consistent naming conventions.
- All public classes and methods shall carry comments describing their purpose, parameters, and return values, per the "well-commented code" deliverable requirement (Section 8).
- Unit tests shall accompany the Domain Layer classes, particularly the RecommendationEngine and its Strategy implementations, to guard against regressions when new strategies are added.

6.7 Portability

Because the stack (MySQL, Python or C++, and a browser-based or terminal-based UI) is cross-platform, the system shall be runnable on Windows, Linux, and macOS development machines without platform-specific code beyond standard build tooling.

7 Data Requirements

7.1 Mock Resource Catalogue

A recommendation engine cannot be meaningfully developed or tested without data. The project therefore requires a mock database of at least 50 to 100 dummy courses before development of the engine begins. Each entry must carry, at minimum, the attributes shown below.

Attribute	Type / Format	Example
title	Text	"Advanced Data Structures"
tags	List of keywords	["data structures", "algorithms", "trees"]
difficulty_level	Enum: Beginner / Intermediate / Advanced	"Advanced"
prerequisites	List of tag/skill references	["Intro to Programming", "Intro to Databases"]
average_rating	Derived, 1.0–5.0	4.2

A representative starter set of titles to seed the mock catalogue includes, for example: "Intro to Databases", "Advanced Data Structures", "Object-Oriented Programming Fundamentals", "Introduction to Machine Learning", "Web Development with JavaScript", "Discrete Mathematics for Computer Science", "Operating Systems Concepts", "Software Testing and QA", and "Cloud Computing Foundations", each expanded with tag and prerequisite metadata to reach the required 50–100 entries.

7.2 Dummy Users and Interaction Histories

In parallel with the resource catalogue, the team shall simulate a set of dummy user profiles with mock interaction histories (past recommendations and ratings). This seed data is essential to verify that the algorithm behaves correctly — for example, that a Beginner-profile learner never receives an Advanced-only recommendation (see TC-05 in Section 9), and that the cold-start fallback activates correctly for a newly created, empty profile.

Dummy Persona	Skills	Goal	Expected Recommendation Behaviour
Persona A — Complete Beginner	None entered	"Learn programming basics"	Cold-start / popularity-based ranking; only Beginner-level resources.
Persona B — Intermediate Learner	"Intro to Programming", "Intro to Databases"	"Become a backend developer"	Personalised ranking; Intermediate resources whose prerequisites are satisfied.

Dummy Persona	Skills	Goal	Expected Recommendation Behaviour
Persona C — Advanced Learner	"Data Structures", "Algorithms", "Databases"	"Prepare for technical interviews"	Personalised ranking; Advanced resources prioritised, with prior ratings influencing order.

8 Project Deliverables

For a Software Engineering laboratory assignment, the source code is only half of the submission. The following comprehensive project report and artefacts shall accompany the code:

- Software Requirements Specification (SRS) — this document, formally documenting functional and non-functional requirements.
- UML Use Case Diagram — depicting Learner and Administrator interactions, per Section 5.5.
- UML Class Diagram — depicting the database schema and OOP classes, per Section 5.4.
- UML Sequence Diagram — depicting the data flow of a user requesting a course recommendation, per Section 5.6.
- UML Activity Diagram — depicting the logic flow of the recommendation algorithm, per Section 5.7.
- Source Code — the functional MVP, well-commented, organised by architectural layer (Section 5.2).
- Test Cases — documentation proving the system was tested, including the cases in Section 9.
- Seed / Mock Data Scripts — SQL or code scripts that populate the 50–100 dummy resources and dummy user personas (Section 7).
- Setup / Run Instructions — a short README enabling a grader to reproduce the environment and demonstration.

9 Test Plan and Representative Test Cases

9.1 Testing Approach

Testing shall combine unit tests for the Domain Layer (particularly the RecommendationEngine and its Strategy implementations), integration tests for the Service and Persistence layers, and manual/scripted acceptance tests against the use cases defined in Section 3. Each test case below is traceable to a functional requirement listed in Section 3.6.

9.2 Representative Test Cases

TC ID	Related Req.	Test Description	Expected Result
TC-01	FR-1 / UC-01	Register with an email already in use.	System rejects registration and displays an "email already registered" error; no duplicate user row is created.
TC-02	FR-1 / UC-02	Log in with an incorrect password three times.	System denies access each time and, after the configured threshold, locks the account or throttles further attempts.
TC-03	FR-2 / UC-03	Save a profile with no skills and no goals selected.	System blocks the save and prompts the user to enter at least one skill or goal.
TC-04	FR-3 / UC-05	Request recommendations for Persona B (Intermediate).	System returns exactly 5 ranked resource IDs, all of Beginner or Intermediate difficulty whose prerequisites are satisfied, within 2 seconds.
TC-05	FR-6 / UC-06	Select the "Beginner" difficulty filter and inspect results.	No resource with difficulty_level = "Advanced" or "Intermediate" appears in the filtered list.
TC-06	FR-7 / UC-05	Request recommendations for Persona A (empty profile).	System activates the cold-start fallback and returns a popularity-ranked list flagged as "cold start".
TC-07	FR-4 / UC-07	Submit a 5-star rating for a recommended resource, then submit again for the same recommendation instance.	The second submission updates the existing rating rather than creating a duplicate row; the resource's average rating recalculates correctly.
TC-08	FR-4 / UC-07	Submit a rating value of 0 or 6.	System rejects the value as out of range (valid range is 1–5) and displays a validation message.

TC ID	Related Req.	Test Description	Expected Result
TC-09	FR-5 / UC-08	Administrator deletes a resource that already has ratings attached.	System prompts for confirmation before cascading the delete, per the exception flow in UC-08.
TC-PERF-01	NFR-1	Run 100 consecutive recommendation requests against the full 50–100 resource catalogue and measure response time.	The 95th-percentile response time is under 2 seconds.
TC-10	NFR-2	Add a new RecommendationStrategy implementation (e.g., a weighted-tag strategy) and switch the engine's configured strategy.	No changes are required to RecommendationService, the persistence schema, or the UI; the new strategy's ranking is reflected end to end.

9.3 Acceptance Criteria

The MVP shall be considered acceptance-ready when: all High-priority use cases in Section 3.6 pass their associated test cases; the performance test (TC-PERF-01) meets the 2-second budget; and no High-severity defect (e.g., data corruption, authentication bypass, or a recommendation that violates the difficulty filter) remains open.

10 Appendices

10.1 Appendix A — Glossary

See Section 1.3 for the primary Definitions, Acronyms, and Abbreviations table. Additional working terms used informally across the team during development include "cold start" (a user with insufficient profile data for personalised ranking) and "catalogue eligibility" (the subset of resources whose prerequisites a given profile satisfies).

10.2 Appendix B — Requirements Traceability Matrix

Req. ID	Source (Problem Statement)	Use Case(s)	Test Case(s)
FR-1	"User Registration/Login" (Functional Requirements)	UC-01, UC-02	TC-01, TC-02
FR-2	"Profile Management" (Functional Requirements)	UC-03, UC-04	TC-03
FR-3	"Recommendation Engine" (Functional Requirements)	UC-05	TC-04, TC-PERF-01
FR-4	"Feedback Loop" (Functional Requirements)	UC-07	TC-07, TC-08
FR-5	Implied by "Mock Database" data requirement	UC-08	TC-09
FR-6	Sample test case in problem statement ("Beginner ⇒ no Advanced")	UC-06	TC-05
NFR-1	"Response Time" (Non-Functional Requirements)	UC-05	TC-PERF-01
NFR-2	"Extensibility" (Non-Functional Requirements)	n/a — architectural	TC-10

10.3 Appendix C — Sprint Plan Outline (Agile Implementation)

Sprint	Focus	Key Outputs
Sprint 0	Requirements & Design	This SRS; UML diagrams; environment setup; MySQL schema.
Sprint 1	Data & Auth Foundation	Mock catalogue (50–100 resources) seeded; User registration/login (UC-01, UC-02).
Sprint 2	Profile & Core Engine	Profile Management (UC-03, UC-04); first RecommendationStrategy; UC-05.
Sprint 3	Feedback & Refinement	Rating/Feedback Loop (UC-07); difficulty filter (UC-06); cold-start fallback.

Sprint	Focus	Key Outputs
Sprint 4	Admin, Testing & Hardening	Catalogue administration (UC-08); test case execution (Section 9); performance tuning; report finalisation.

10.4 Appendix D — Assumptions Log

This appendix consolidates the assumptions stated throughout the document (Section 2.6) for quick reference during review: a mock/dummy dataset is acceptable in place of a live licensed catalogue; a local or lightweight cloud MySQL instance is available throughout development; authentication may be simplified to username/password for the MVP; and payment, live third-party integrations, native mobile apps, and social features are out of scope for this release.

— End of Document —